Hito 2

Consolidación, Validación y Despliegue en Producción

CleanPool App

Equipo: Cleanpool

Sección: 300

Entrega: 29 de abril de 2026

Integrantes: Vicente Araya (PM) | Diego Carmona (SM) | Daniel Cortez | Constanza Malebran

Índice

1. Exposición del Problema y Propuesta de Valor

1.1. Definición del Problema

Describir el problema central que el sistema busca resolver. Explicar por qué existe, a quién afecta y cuál es su magnitud o frecuencia. Ser concreto y evitar definiciones genéricas.

La gestión de la calidad del agua en piscinas residenciales y de pequeña escala suele depender de mediciones manuales esporádicas, lo que dificulta detectar desviaciones de pH, cloro o temperatura a tiempo. Esto afecta a los dueños porque incrementa el riesgo sanitario, el sobreuso de químicos y los costos de mantención. El sistema aborda este problema mediante monitoreo continuo con sensores IoT y recomendaciones accionables desde una aplicación cliente.

1.2. Contexto y Usuarios Objetivo

- **Usuarios principales:** Propietarios particulares de piscinas que desean mantener el agua de sus piscinas en perfectas condiciones, pero que no poseen conocimientos técnicos acerca del mantenimiento en piscinas.
- **Entorno de uso:** Aplicación Flutter multiplataforma conectada a una API FastAPI y MongoDB Atlas, con ingesta de sensores desde dispositivos ESP mediante endpoints protegidos por API key.
- **Necesidad identificada:** Contar con visibilidad continua del estado del agua y soporte de decisiones para tratamiento manual, reduciendo dependencia de chequeos reactivos.

1.3. Propuesta de Valor y Evidencia

Elemento	Descripción
Caso de uso 1	Usuario autenticado consulta estado de su piscina (pH, cloro, temperatura) a partir de lecturas recientes ingresadas manualmente.
Caso de uso 2	Usuario registra tratamiento manual y revisa historial de mantenciones para trazabilidad operativa.
Caso de uso 3	Usuario obtiene temperatura de su piscina en tiempo real a través de sensor de temperatura.
Impacto esperado	El usuario puede mantener en óptimas condiciones el agua de su piscina, además de mejorar la consistencia en decisiones de mantención.
Validación preliminar	El usuario obtiene temperatura de su piscina y además consigue conocimientos necesarios para la mantención de su piscina.

2. Requerimientos Funcionales y No Funcionales

2.1. Requerimientos Funcionales

ID	Nombre	Descripción	Prioridad
Acceso y seguridad de usuarios			
RF-01	Autenticación de usuarios	El sistema debe permitir registro, inicio de sesión y consulta de perfil autenticado mediante JWT.	Alta
RF-15	Ingreso seguro propietario	El sistema debe implementar autenticación segura para el ingreso del propietario de la piscina.	Alta
Gestión de piscinas y dispositivos IoT			
RF-02	Gestión de piscinas	El sistema debe permitir crear, listar, actualizar y eliminar piscinas asociadas al usuario autenticado.	Alta

ID	Nombre	Descripción	Prioridad
RF-05	Ingesta y vínculo de dispositivos	El sistema debe recibir telemetría IoT protegida por API key y permitir vincular/desvincular dispositivos a piscinas.	Media
RF-13	Estado al vincular IoT	El sistema debe mostrar el estado del agua cada vez que se vincule el dispositivo (IoT) a la app.	Media
Medición, evaluación y visualización del estado del agua			
RF-06	Medición de temperatura	El sistema debe medir temperatura del agua.	Alta
RF-07	Comparación con rangos óptimos	El sistema debe comparar automáticamente los valores medidos con rangos óptimos predefinidos para piscinas residenciales.	Alta

ID	Nombre	Descripción	Prioridad
RF-08	Indicadores fuera de rango	El sistema debe generar indicadores cuando uno o más parámetros estén fuera de rango seguro, indicando el parámetro afectado con su valor.	Alta
RF-03	Monitoreo de estado	El sistema debe evaluar lecturas de sensores y entregar estado por parámetro y estado global de aptitud.	Alta
RF-12	Aptitud para baño	El sistema debe mostrar en el dashboard si el agua está apta o no apta para el baño, basándose en la evaluación conjunta de los parámetros medidos.	Alta

ID	Nombre	Descripción	Prioridad
RF-11	Dashboard móvil	El sistema debe permitir al usuario consultar el estado actual de la piscina desde un dashboard dentro de una app móvil, accesible desde cualquier smartphone.	Alta
RF-10	Historial de mediciones	El sistema debe almacenar el historial de mediciones con identificador de sensor y valor registrado.	Alta
Recomendación y acciones de mantención			
RF-09	Recomendación química	El sistema debe recomendar qué producto químico aplicar y en qué cantidad aproximada según volumen de piscina y valores actuales.	Alta

ID	Nombre	Descripción
Prioridad		
RF-14	Acción recomendada	El sistema debe mostrar la acción recomendada en base a los parámetros del agua.
Alta		
RF-04	Mantenciones y tratamientos	El sistema debe calcular dosis de tratamiento manual y persistir historial de mantenciones.
Media		

2.2. Requerimientos No Funcionales

ID	Categoría	Descripción
Métrica		
Seguridad		
RNF-03	Seguridad	Las credenciales deben almacenarse hasheadas y el acceso a endpoints protegidos debe validarse con token Bearer JWT; la ingesta IoT exige API key.
JWT + bcrypt + X-API-KEY		

ID	Categoría	Descripción
Métrica		
RNF-10	Seguridad	El acceso a los datos debe requerir autenticación con token JWT.
JWT obligatorio		
Disponibilidad e infraestructura		
RNF-02	Disponibilidad	El backend debe ser desplegable en entorno productivo y ejecutable localmente por Docker o Uvicorn.
RNF-07	Disponibilidad	El sistema debe estar continuamente disponible un 90 % del tiempo a nivel semanal garantizando su estabilidad.
90 % semanal		

ID	Categoría	Descripción
Métrica		
RNF-15	Disponibilidad en nube	El sistema debe estar desplegado en infraestructura cloud que garantice una disponibilidad mínima del 97 % mensual, excluyendo ventanas de mantenimiento programadas.
97 % mensual		
Rendimiento y latencia		
RNF-01	Rendimiento	La API debe responder de forma interactiva para operaciones de consulta y registro sobre lecturas, piscinas y auth.
RNF-09	Latencia de actualización	La visualización de datos en el dashboard debe actualizarse en al menos 5 segundos desde la captura del sensor.
$\leq$ 5 segundos		

ID Métrica	Categoría	Descripción
RNF-13 $\leq$ 5 segundos	Rendimiento funcional	El sistema debe mostrar recomendaciones de productos químicos en menos de 5 segundos una vez que se hace clic en solicitar la recomendación.
Calidad de datos y precisión		
RNF-08 Error $<$ $\pm 1\,^{\circ}\mathrm{C}$	Precisión	Las mediciones de sensores tienen un margen de error menor a ± 1 en temperatura.
RNF-12	Validación de datos	El sistema debe validar cada lectura antes del almacenamiento, aceptando solo pH entre 0.0 y 14.0, temperatura entre 0°C y 60°C y cloro entre 0.0 y 10.0 ppm; las lecturas fuera de rango se descartan.
Usabilidad y compatibilidad		

ID	Categoría	Descripción
Métrica		
RNF-04	Usabilidad	La interfaz Flutter debe presentar navegación por pantallas de autenticación, dashboard, perfil y gestión de piscina con flujo comprensible para usuario final.
RNF-06	Usabilidad	La interfaz debe mostrar los niveles de pH, cloro y temperatura al hacer clic en dashboard.
Visualización en dashboard		
RNF-14	Compatibilidad móvil	La app debe ser completamente funcional en dispositivos iOS versión 14.0 o superior y Android 10.0 o superior.
iOS 14+ y Android 10+		
Mantenibilidad y evolución		

ID Métrica	Categoría	Descripción
RNF-11	Mantenibilidad	El sistema debe diseñarse de forma modular con capas separadas en IoT, lógica de negocio, datos y presentación para facilitar correcciones y mejoras sin afectar otros módulos.
Arquitectura en capas		
RNF-05	Escalabilidad	La arquitectura desacoplada frontend/backend y uso de MongoDB Atlas permite crecer en clientes y lecturas sin acoplar UI con almacenamiento.

3. Diseño y Justificación Arquitectónica

3.1. Arquitectura del Sistema

La arquitectura se compone de: (1) cliente Flutter (móvil) como capa de presentación; (2) API REST FastAPI como capa de aplicación; (3) MongoDB Atlas como persistencia; y (4) integración de ingesta IoT mediante endpoints versionados con autenticación por API key para dispositivos. El backend expone además autenticación JWT para usuarios finales y servicios de negocio para evaluación de estado y mantenciones.

3.2. Cambios Respecto al Hito Anterior

Componente	Estado anterior	Estado actual
Backend API	Endpoints base de autenticación/lecturas y estructura inicial.	Se consolidan routers para ingesta IoT, vinculación de dispositivos, piscinas de usuario y mantenciones.
Cliente Flutter	Flujo inicial de interfaz y consumo básico de API.	Se estructura por features con servicios compartidos para auth, pool y mantenciones, incluyendo persistencia local de sesión.
Placa arduino	Uso de arduino ESP32	Uso de arduino ESP8266
Access point desde esp82666 para conexión con la app	esp8266 se conecta a red Wifi y asi manda datos con protocolo HTTP al backend de la app	

3.3. Decisiones de Diseño y Trade-offs

Decisión	Justificación	Trade-off asumido
REST (FastAPI) vs GraphQL	Menor complejidad operativa y rapidez de implementación para endpoints orientados a recursos del dominio.	

Decisión	Justificación	Trade-off asumido
	Simplicidad y estandarización HTTP, a cambio de menor flexibilidad en agregación de consultas complejas.	
	Monolito modular vs microservicios	
	Tamaño del equipo y del dominio favorecen una base única con routers/servicios separados por responsabilidad.	
	Menor costo de despliegue y coordinación, a cambio de mayor riesgo de crecimiento acoplado si no se controla deuda técnica.	

3.4. Problemas Detectados y Soluciones

Problema detectado	Solución implementada
No se pudo hacer funcionar el sensor de ph ya que era imposible de calibrar.	Se mantiene sensor de temperatura funcionando
Inconsistencia entre rutas históricas y rutas actuales en pruebas heredadas.	Se mantiene cobertura principal con tests específicos por módulo (pools, mantenciones, status) y se documenta el desfase para normalizar endpoints.
El arduino no lograba enviar los datos del sensor de temperatura a la aplicación	Se modificó el endpoint necesario apuntando al backend desplegado para mostrar temperatura en aplicación

El modelado 3D del diseño al principio generó problemas de impresión que hicieron que no se pudiera imprimir por ancho de paredes	Se ajustaron las medidas del modelo del flotador en tinkercad para ser impreso en 3D

4. Estrategia de Pruebas

4.1. Enfoque General

Se adopta un enfoque mixto con pruebas unitarias y de integración en backend usando Pytest, FastAPI TestClient y mocks de MongoDB para desacoplar dependencias externas. En frontend existe prueba base de widget para validar arranque. La priorización se centra en autenticación, estado de piscina, mantenciones y configuración de pools.

4.2. Plan de Pruebas

ID	Tipo	Módulo/Función	Descripción	Resultado
TP-01	Unitaria	status_service	Valida cálculo de estado global de aptitud desde estados de parámetros.	*[Implementada en repositorio]*
TP-02	Unitaria	pools/settings	Verifica comportamiento de defaults, límites y cálculo de estado para configuración de piscina.	*[Implementada en repositorio]*

ID	Tipo	Módulo/Función	Descripción	Resultado
TP-03	Integración	auth + mantenciones	Comprueba flujo autenticado para crear/listar mantenciones y restricciones sin token.	*[Implementada en repositorio]*
TP-04	Integración	test_api legado	Incluye casos sobre /readings que no coinciden completamente con routers activos en main.	*[Requiere alineación]*
TP-05	Sistema	frontend widget test	Valida construcción básica de la app Flutter en entorno de pruebas.	*[Implementada en repositorio]*

4.3. Cobertura y Herramientas

- **Herramientas utilizadas:** Pytest, FastAPI TestClient, unittest.mock (backend) y flutter_test (frontend).
- **Cobertura de código alcanzada:** *[Sin reporte de cobertura cuantitativa en repositorio.]*
- **Criterio de aceptación:** Los módulos críticos (auth, estado de piscina, mantenciones, pools) deben contar con pruebas automatizadas y mantenerse operativos en integración API.

5. Identificación y Gestión de Riesgos

5.1. Matriz de Riesgos

ID	Descripción del riesgo	Probabilidad	Impacto	Plan de mitigación	Estado
R-01	Dependencia de servicios externos (MongoDB Atlas / nube) para operación continua.	Media	Alto	Mantener variables de entorno claras, validación de conexión y plan de contingencia para entorno alternativo/local.	Activo
R-02	Desfase entre rutas legacy y rutas vigentes en pruebas/documentación.	Media	Medio	Homologar contratos API, depurar endpoints en desuso y actualizar suite de pruebas.	Activo
R-03	CORS completamente abierto en backend en un eventual despliegue público.	Media	Medio	Restringir orígenes permitidos y endurecer configuración por entorno.	Activo

ID	Descripción del riesgo	Probabilidad	Impacto	Plan de mitigación	Estado
R-04	Deuda técnica por duplicidad de modelos/colecciones (piscinas/pools, mantenciones/mantenimientos).	Media	Medio	Planificar refactor de unificación de dominio y normalización de repositorios.	Activo

5.2. Riesgos Materializados y Respuesta

Se observa materialización parcial del riesgo de inconsistencia técnica entre componentes legacy y actuales (por ejemplo, rutas probadas históricamente versus routers incluidos en la aplicación). El impacto principal es aumento de costo de mantenimiento y ambigüedad documental. Como respuesta, se prioriza normalización de contratos API, consolidación de nomenclatura de colecciones y actualización de pruebas de integración.

6. Incremento Funcional y Despliegue

6.1. Funcionalidades Implementadas en este Hito

ID	Historia de usuario	Descripción	Estado
HU-01	Como usuario, quiero registrarme e iniciar sesión para acceder de forma segura a mis datos de piscina.	Implementación en endpoints /auth/register, /auth/login, /auth/me y persistencia local de sesión en Flutter.	**Completa**

ID	Historia de usuario	Descripción	Estado
HU-02	Como usuario, quiero visualizar el estado de mi piscina para decidir acciones de mantención.	Evaluación de lecturas y estado global, además de endpoints de consulta y pantallas de dashboard/perfil.	**Completa**
HU-03	Como usuario, quiero que el dispositivo quede vinculado de forma trazable a mi piscina para automatizar lecturas.	Existen endpoints de bind/unbind/status, pero persisten elementos legacy y normalización pendiente entre modelos de dominio.	**Parcial**

6.2. Entorno de Despliegue

Parámetro	Detalle
Plataforma	Azure Web App (backend) y despliegue frontend web por configuración Docker/Nginx y Vercel SPA config en repositorio.
URL de acceso	*[No visible explícitamente en el repositorio]*
Credenciales de prueba	*[No deben publicarse; no hay credenciales de demo explícitas en repositorio]*
Estado al momento de entrega	Backend con pipeline de despliegue configurado y ejecución local documentada para backend/frontend.
Notas adicionales	Endpoints de usuario requieren Bearer JWT; ingesta de sensores requiere header X-API-KEY.

6.3. Evidencia de Funcionamiento

Incluir capturas de pantalla o descripciones de las funcionalidades clave en el entorno de producción. Si se incluyen imágenes, descomentar el bloque `figure` y reemplazar el nombre del archivo.

Se puede evidenciar el flujo completo: registro/login, visualización de estado de piscina en dashboard, ejecución de tratamiento manual y revisión de historial de mantenciones, además del registro de datos de sensores en backend vía endpoints de ingesta.

7. Aplicación de la Metodología Scrum

7.1. Gestión del Sprint (Product Manager)

Parámetro	**Detalle**
Número de sprint	*[Sprint #X]*
Objetivo del sprint	*[Descripción del objetivo comprometido.]*
Grado de cumplimiento	*[ej. 80 % — 4 de 5 historias completadas.]*
Ajustes realizados	*[Cambios en el alcance o prioridad y su justificación.]*
Objetivo del siguiente sprint	*[Descripción del sprint planificado.]*

Estado del Product Backlog

ID	**Historia de usuario**	**Sprint asignado**	**Estado**
HU-01	*[descripción]*	Sprint 1	**Completa**
HU-02	*[descripción]*	Sprint 2	**Completa**
HU-03	*[descripción]*	Sprint 2	**Parcial**
HU-04	*[descripción]*	Sprint 3	Pendiente
HU-05	*[descripción]*	Sprint 3	Pendiente

7.2. Ejecución Scrum (Scrum Master)

Daily Meetings

[Describir la dinámica de los Daily Meetings: frecuencia, modalidad (presencial/remota), duración promedio y disciplina del equipo.]

Sprint Retrospective — Principales Mejoras

Aspecto	Acción tomada
[Problema identificado]	*[Mejora implementada]*
[Problema identificado]	*[Mejora implementada]*

Impedimentos y Adaptaciones del Equipo

[Describir los principales impedimentos encontrados durante el sprint y cómo el equipo los resolvió o mitigó.]

7.3. Registro de Asistencia a Daily Meetings

Nota: Este componente tiene alto peso evaluativo. El registro debe ser completo, individual y reflejar la participación real del equipo.

Integrante	D1	D2	D3	D4	D5	D6	Asistidas	%
Integrante 1 (PM)	✓	✓	✓	×	✓	✓	5/6	83 %
Integrante 2 (SM)	✓	✓	✓	✓	✓	✓	6/6	100 %
Integrante 3	✓	×	✓	✓	×	✓	4/6	67 %
Integrante 4	✓	✓	✓	✓	✓	×	5/6	83 %

Justificación de inasistencias:

- **Integrante 1, D4:** *[Motivo justificado.]*
- **Integrante 3, D2 y D5:** *[Motivo justificado.]*

8. Presentación Final Orientada a Stakeholders

8.1. Narrativa del Proyecto

CleanPool App aborda el problema de control reactivo de calidad de agua en piscinas, entregando monitoreo digital y apoyo a decisiones de mantención para usuarios responsables

de operación. En este hito se consolidó una arquitectura operativa con frontend Flutter, backend FastAPI y persistencia en MongoDB Atlas, incorporando autenticación, gestión de piscinas, ingesta IoT, evaluación de estado y registro de mantenciones. El valor entregado es una base funcional desplegable que centraliza información crítica y mejora la trazabilidad de acciones técnicas.

8.2. Avances Clave del Hito

- Se implementó flujo de autenticación completo con sesión persistente en cliente Flutter.
- Se habilitó monitoreo del estado del agua con evaluación por parámetros y estado global.
- Se incorporó gestión de mantenciones y tratamiento manual con historial consultable.

8.3. Demostración en Producción

El sistema se encuentra desplegado y disponible para evaluación en: ***[URL no especificada en repositorio]***.
Credenciales de acceso para la demo: **Usuario:** *[No disponible en repositorio]* **Contraseña:** *[No disponible en repositorio]*

Flujo sugerido de demostración: (1) registro/inicio de sesión, (2) creación/selección de piscina, (3) revisión de estado actual y métricas, (4) ejecución de tratamiento manual, (5) validación de historial de mantenciones y (6) evidencia de endpoints de ingesta/vinculación de dispositivo.

8.4. Próximos Pasos

1. Normalizar y unificar modelos de dominio duplicados entre rutas legacy y rutas vigentes.
2. Alinear pruebas, documentación y contratos de API para eliminar rutas obsoletas y asegurar trazabilidad funcional.
3. Endurecer configuración de seguridad y observabilidad (CORS por entorno, logging estructurado, reporte de cobertura).

Anexo A. Capturas de Pantalla del Sistema

[Pendiente adjuntar capturas reales del entorno desplegado.]

Anexo B. Actas de Daily Meetings

[Pendiente adjuntar actas reales del equipo.]

Anexo C. Evidencia de Repositorio

Recurso	**URL**
Repositorio (GitHub / GitLab)	*[URL exacta del repositorio no especificada en este archivo]*
Tablero Scrum (Jira / Trello)	*[No evidenciado en repositorio]*
Sistema desplegado	*[URL pública no evidenciada en repositorio]*